Fases, temas, tareas y rúbricas

Guía integrada para ComprasLab - Proyecto de Agentes Cognitivos con LangChain

Objetivo del documento. Reunir en una sola guía las cinco fases solicitadas, los temas de las clases 2 a 6 que debes demostrar, las tareas concretas que debes ejecutar y las rúbricas que evidencian el cumplimiento.

Cómo usarla. Trabaja fase por fase. No marques una rúbrica como “cumplida” solo porque existe código: conserva una ejecución que demuestre el comportamiento y una explicación de la decisión técnica.

Mapa rápido

Fase	Rúbricas	Resultado principal
1	R01-R06	Chain ejecutable y arquitectura
2	R07-R12	Agente con herramientas y ReAct
3	R13-R18	Chatbot con memoria
4	R19-R23	SQL, APIs y RAG existente
5	R24-R29	Agente final verificado y evaluado

Fuente académica. Las cinco fases y las preguntas de rúbrica se tomaron del enunciado del proyecto y su rúbrica (páginas 2-8). Los temas T1-T26 proceden del resumen “Temas de las clases 2 a 6”. Las adaptaciones a ComprasLab son decisiones de diseño del proyecto.

Fase 1. De la idea al MVP: chain ejecutable con entrada y salida estructuradas

Objetivo: Construir la base del sistema: definir la arquitectura del agente y crear una primera cadena que funcione de punta a punta con entrada y salida previsibles.

Temas que debes demostrar y rúbricas relacionadas

Tema	Contenido de clase	Rúbrica
T1	Configuración de LangChain y ChatOpenAI.	R01
T2	PromptTemplate y ChatPromptTemplate.	R03
T3	Salidas estructuradas con Pydantic / structured output.	R04
T4	Runnable: invoke(), batch() y stream().	R05
T5	Composición con @chain y LCEL.	R05-R06
T6, T17, T18, T21	Orquestación, memoria y descomposición para diseñar arquitectura y planificación.	R01-R02

Tareas que debes ejecutar

1. Describir la anatomía de ComprasLab: LLM, system message, herramientas previstas, memoria, base de conocimiento y flujo general.

2. Clasificar el agente como reactivo, deliberativo o híbrido y documentar su madurez actual y su madurez objetivo.

3. Diseñar, todavía a nivel de arquitectura, cómo se dividirán las solicitudes complejas y qué información irá a memoria de corto y largo plazo.

4. Crear PromptTemplate y/o ChatPromptTemplate reutilizables para interpretar solicitudes de compra.

5. Crear una cadena que transforme una petición como “20 monitores, R$12.000, 7 días” en una estructura JSON validada.

6. Demostrar invoke(), batch() y stream() en ejemplos reproducibles.

7. Implementar una versión imperativa con @chain y otra declarativa con LCEL; comparar ambas y justificar la elegida como patrón principal.

8. Guardar capturas o salidas del notebook que prueben entrada, pasos relevantes y salida final.

Aplicación en ComprasLab

En ComprasLab, esta fase convierte una solicitud de compra en una estructura confiable con producto, cantidad, presupuesto, plazo y datos faltantes. Todavía no necesitas el agente completo: debes demostrar que la base funciona y que la arquitectura futura está bien pensada.

Fase 1 - Rúbricas que debes demostrar

R01 - ¿Estructuró la anatomía y la arquitectura cognitiva del agente, clasificando su tipo y nivel de madurez agéntica?
Evidencia esperada: Diagrama/explicación de componentes, clasificación del agente y madurez actual/objetivo.

R02 - ¿Proyectó una arquitectura de planificación y memoria aumentada para el agente?
Evidencia esperada: Diseño de descomposición de tareas y estrategia de memoria del dominio.

R03 - ¿Construyó plantillas de prompt reutilizables con PromptTemplate o ChatPromptTemplate?
Evidencia esperada: Código y ejecución de plantillas con variables del dominio.

R04 - ¿Implementó un output parser para generar salida estructurada en JSON a partir de la chain?
Evidencia esperada: JSON/objeto validado producido desde una solicitud real.

R05 - ¿Aplicó Runnable (invoke, batch, stream) e implementó composición imperativa con @chain?
Evidencia esperada: Ejecuciones demostrables de los tres métodos y una chain con @chain.

R06 - ¿Implementó composición declarativa con LCEL y justificó la elección entre composición imperativa y declarativa?
Evidencia esperada: Código LCEL + comparación técnica escrita.

Checklist de cierre de la fase

• El código o notebook se ejecuta desde cero con instrucciones reproducibles.

• Existe al menos una ejecución que demuestra cada rúbrica de esta fase.

• La evidencia diferencia claramente datos reales, datos dummy y resultados simulados.

• La documentación explica qué decisión técnica tomaste y por qué.

• Los pendientes o limitaciones están escritos explícitamente; no se presentan como funcionalidades terminadas.

Fase 2. Del prototipo al agente: herramientas, ReAct y prompts avanzados

Objetivo: Transformar la chain inicial en un agente que decide qué hacer y puede utilizar herramientas reales fuera del propio prompt.

Temas que debes demostrar y rúbricas relacionadas

Tema	Contenido de clase	Rúbrica
T6	Orquestación: flujo predefinido vs. agente que decide.	R07-R08
T7	Comparación de LangChain, LangGraph, LlamaIndex, AutoGen, Semantic Kernel y Langflow.	R07
T8	Cadenas secuenciales y procesamiento en paralelo.	R09
T9	Enrutamiento dinámico.	R09
T10	@tool y function calling con API externa.	R10
T11	Agente ReAct y memoria de sesión.	R08
T12	Few-shot prompting.	R11
T13-T16	Parsers, manejo de errores, memoria, FAISS y callbacks.	R12

Tareas que debes ejecutar

1. Comparar al menos dos orquestadores por autonomía, abstracción y modularidad; justificar por qué LangChain se mantiene en ComprasLab.

2. Construir una demostración ReAct con initialize_agent y ZERO_SHOT_REACT_DESCRIPTION, usando herramientas y memoria básica.

3. Implementar cadenas secuenciales y paralelas donde aporten valor; mostrar qué etapas pueden ejecutarse de forma independiente.

4. Implementar routing dinámico con RouterChain y demostrar entradas que toman rutas distintas.

5. Integrar por lo menos una API externa real como herramienta mediante function calling. En ComprasLab se propone ViaCEP para confirmar ciudad/UF del destino.

6. Diseñar el prompt del agente con PTCF (Persona, Task, Context, Format) e incluir ejemplos few-shot del dominio.

7. Documentar descomposición de tareas y decisiones observables del agente sin depender de razonamiento privado.

8. Añadir componentes de confiabilidad: parser con tratamiento de error, memoria configurada, búsqueda FAISS cuando corresponda y callbacks de logging.

9. Conservar trazas o logs que permitan comprobar qué herramienta fue solicitada, con qué argumentos y qué resultado devolvió.

Aplicación en ComprasLab

ComprasLab pasa de “transformar texto” a “decidir y actuar”: buscar un producto, consultar ofertas, usar una API o recuperar una política según lo que necesite la solicitud.

Fase 2 - Rúbricas que debes demostrar

R07 - ¿Comparó orquestadores de agentes autónomos y justificó la elección de LangChain para el problema?
Evidencia esperada: Tabla comparativa y decisión técnica argumentada.

R08 - ¿Construyó un agente con herramientas y memoria e implementó el patrón ReAct con initialize_agent?
Evidencia esperada: Ejecución ReAct real mostrando elección y uso de herramienta.

R09 - ¿Construyó chains secuenciales o paralelas e implementó routing dinámico con RouterChain?
Evidencia esperada: Dos o más rutas demostradas y composición de chains.

R10 - ¿Integró al menos una API externa como herramienta del agente vía function calling?
Evidencia esperada: Llamada real con argumentos, respuesta de API y uso del resultado.

R11 - ¿Estructuró el prompt con PTCF y aplicó few-shot learning y chain-of-thought?
Evidencia esperada: Prompt documentado, ejemplos few-shot y planificación/justificación observable.

R12 - ¿Implementó componentes de confiabilidad: output parsers con errores, memoria, FAISS y callbacks, justificando las elecciones?
Evidencia esperada: Casos normales y casos de error, más logs y evidencia de recuperación.

Checklist de cierre de la fase

• El código o notebook se ejecuta desde cero con instrucciones reproducibles.

• Existe al menos una ejecución que demuestra cada rúbrica de esta fase.

• La evidencia diferencia claramente datos reales, datos dummy y resultados simulados.

• La documentación explica qué decisión técnica tomaste y por qué.

• Los pendientes o limitaciones están escritos explícitamente; no se presentan como funcionalidades terminadas.

Fase 3. Memoria de corto y largo plazo: el agente se convierte en chatbot

Objetivo: Añadir continuidad entre turnos y sesiones para que el agente recuerde contexto relevante, recupere recuerdos y gestione conversaciones complejas.

Temas que debes demostrar y rúbricas relacionadas

Tema	Contenido de clase	Rúbrica
T14	Buffer, límite de tokens y resumen del historial.	R14, R16
T17	Memoria corta y larga: semántica, episódica y procedimental.	R13
T18	Ventana de contexto, relevancia, antigüedad y frecuencia.	R14
T19	Mem0, Letta/MemGPT y ejercicio inspirado en LangMem.	R15
T20	Intenciones, entidades, datos faltantes y reutilización de turnos previos.	R16-R18
T21	Descomposición de solicitudes complejas en varias acciones.	R18

Tareas que debes ejecutar

1. Definir qué información de ComprasLab pertenece a memoria de trabajo, semántica, episódica y procedimental usando la taxonomía CoALA.

2. Elegir una estrategia de ventana de contexto: sliding window o resumen, y combinarla con recuperación vectorial de recuerdos relevantes.

3. Definir reglas para actualizar, reemplazar o ignorar recuerdos obsoletos.

4. Comparar al menos dos herramientas especializadas de memoria, por ejemplo LangMem y Mem0, y justificar la elección o la implementación propia.

5. Construir una demostración de chatbot con ConversationBufferMemory y varios turnos.

6. Implementar reconocimiento de intención y extracción de entidades: producto, cantidad, presupuesto, plazo y destino.

7. Pedir explícitamente la información que falte, en lugar de inventarla.

8. Demostrar una conversación multi-turno en la que el usuario modifica requisitos y el agente conserva los que siguen vigentes.

9. Demostrar aislamiento entre sesiones/usuarios y, si afirmas persistencia, probar recuperación después de reiniciar el proceso.

Aplicación en ComprasLab

Ejemplo: “Necesito 20 monitores”. Luego: “Ahora quiero 12 y mantén el plazo”. El agente debe conservar producto/plazo, actualizar

cantidad y volver a evaluar. En otra sesión puede recuperar una preferencia del usuario, sin reutilizar precios obsoletos de otro catálogo.

Fase 3 - Rúbricas que debes demostrar

R13 - ¿Diferenció memoria de corto y largo plazo con base en la taxonomía CoALA?
Evidencia esperada: Mapa de memorias del dominio y ejemplos concretos.

R14 - ¿Gestionó ventanas de contexto combinando sliding window o summarization con búsqueda vectorial?
Evidencia esperada: Ejecución que muestre contexto reducido + memoria recuperada.

R15 - ¿Comparó herramientas especializadas de memoria por arquitectura y caso de uso?
Evidencia esperada: Comparación de dos o más opciones y justificación.

R16 - ¿Construyó un chatbot conversacional con ConversationBufferMemory?
Evidencia esperada: Conversación real de varios turnos con contexto.

R17 - ¿Estructuró flujos de conversación con reconocimiento de intención y extracción de entidades?
Evidencia esperada: Mensajes reales con intención, entidades y aclaraciones.

R18 - ¿Validó, con evidencia de una conversación real, que el agente mantiene contexto y descompone consultas complejas en diálogos multi-turno?
Evidencia esperada: Transcripción/trace de una conversación compleja con actualización de estado.

Checklist de cierre de la fase

• El código o notebook se ejecuta desde cero con instrucciones reproducibles.

• Existe al menos una ejecución que demuestra cada rúbrica de esta fase.

• La evidencia diferencia claramente datos reales, datos dummy y resultados simulados.

• La documentación explica qué decisión técnica tomaste y por qué.

• Los pendientes o limitaciones están escritos explícitamente; no se presentan como funcionalidades terminadas.

Fase 4. Herramientas externas y RAG existente: ampliar el alcance del agente

Objetivo: Conectar el agente con datos estructurados, APIs y un índice documental existente, y demostrar que puedes diagnosticar una recuperación deficiente.

Temas que debes demostrar y rúbricas relacionadas

Tema	Contenido de clase	Rúbrica
T10	APIs mediante herramientas y function calling.	R20
T15-T16	FAISS, búsqueda semántica, metadatos y callbacks.	R22
T22	Herramientas deterministas, semánticas y asíncronas.	R19
T23	Text-to-SQL sobre SQLite.	R21
T24	RAG, chunking, reformulación, filtros y diagnóstico.	R22-R23
T25	Integración SQL + FAISS + herramientas en un agente.	R19-R22
T26	Diagnóstico de búsquedas fallidas y falta de respaldo.	R23

Tareas que debes ejecutar

1. Documentar cada herramienta con nombre, descripción, entradas, salidas, errores y tipo de lógica: hardcoded/determinista, semántica, API o text-to-query.

2. Integrar las APIs internas o externas necesarias para el dominio y explicar por qué cada una existe.

3. Implementar text-to-query: convertir una pregunta natural en SQL de lectura, validarlo, ejecutarlo y usar las filas para responder.

4. Restringir SQL a tablas/columnas permitidas, consultas de lectura y límites de resultados.

5. Localizar y cargar el índice FAISS preexistente exigido por la actividad, junto con su docstore, metadatos y modelo de embeddings compatible.

6. Exponer la recuperación FAISS como herramienta del agente sin reconstruir innecesariamente el pipeline de indexación solicitado como “existente”.

7. Provocar o localizar un fallo real de recuperación en el que la evidencia correcta exista en el corpus pero no sea recuperada adecuadamente.

8. Guardar los fragmentos recuperados antes de la corrección, diagnosticar el problema y aplicar una técnica como query rewriting, múltiples consultas o documento hipotético.

9. Repetir exactamente el caso y comparar la recuperación antes/después con evidencia identificable.

Aplicación en ComprasLab

ComprasLab usa SQL para datos exactos (producto, precio, stock y plazo) y FAISS para políticas/documentos. El objetivo académico no es solo “tener RAG”: debes mostrar un caso en que la recuperación falla y cómo la corriges.

Fase 4 - Rúbricas que debes demostrar

R19 - ¿Estructuró herramientas diferenciando funciones hardcoded de funciones semánticas?
Evidencia esperada: Catálogo de herramientas y justificación de su tipo.

R20 - ¿Integró el agente con APIs internas o externas de acuerdo con el tipo de uso de cada una?
Evidencia esperada: Integración funcional explicando qué aporta cada API.

R21 - ¿Implementó el patrón text-to-query para consultar datos estructurados?
Evidencia esperada: Pregunta natural, consulta generada, validación, ejecución y filas.

R22 - ¿Construyó un agente de recuperación usando un índice FAISS existente?
Evidencia esperada: Carga del índice existente y recuperación mediante herramienta.

R23 - ¿Diagnosticó un fallo real de recuperación a partir de los documentos retornados y justificó la técnica de corrección aplicada?
Evidencia esperada: Comparativa before/after con diagnóstico y evidencia fuente.

Checklist de cierre de la fase

• El código o notebook se ejecuta desde cero con instrucciones reproducibles.

• Existe al menos una ejecución que demuestra cada rúbrica de esta fase.

• La evidencia diferencia claramente datos reales, datos dummy y resultados simulados.

• La documentación explica qué decisión técnica tomaste y por qué.

• Los pendientes o limitaciones están escritos explícitamente; no se presentan como funcionalidades terminadas.

Fase 5. Agente integrador: verificación de respuestas y evaluación con LangSmith

Objetivo: Unificar lo construido en un agente completo, verificar que sus respuestas estén respaldadas y evaluar tanto el resultado final como la trayectoria de herramientas.

Temas que debes demostrar y rúbricas relacionadas

Tema	Contenido de clase	Rúbrica
T16	Callbacks y observabilidad.	Apoyo R29
T24, T26	Evidencia documental y diagnóstico de falta de respaldo.	Apoyo R24
T25	Agente integrador con SQL, FAISS y otras herramientas.	R25
T11, T14, T20	Agentes y memoria: base conceptual para ejecutor e historial.	Apoyo R26-R27
No explícito en T1-T26	NLI, create_openai_tools_agent, AgentExecutor, RunnableWithMessageHistory, Streamlit y LangSmith como requisitos del proyecto.	R24-R29

Tareas que debes ejecutar

1. Construir un mecanismo de verificación que forme parte del flujo del agente, no un análisis manual posterior.

2. Aplicar un modelo NLI para clasificar relación entre afirmación y evidencia como entailment/sustentación, contradiction o neutral.

3. Añadir comprobaciones deterministas de coherencia: total, stock, presupuesto, plazo, IDs y procedencia de fuentes.

4. Consolidar un solo agente con al menos dos categorías de herramientas; en ComprasLab se recomienda SQL + RAG + API.

5. Construir el agente final con create_openai_tools_agent y AgentExecutor, respetando las APIs nombradas por la rúbrica.

6. Integrar historial con RunnableWithMessageHistory y demostrar continuidad de sesión.

7. Integrar el agente completo a Streamlit para cubrir la rúbrica específica, aunque el texto de desarrollo lo presente como opcional.

8. Crear un conjunto de evaluación para LangSmith con respuestas esperadas y trayectorias esperadas de herramientas.

9. Evaluar calidad de respuesta y trayectoria de llamadas; analizar métricas y justificar conclusiones.

10. Conservar evidencias reales: traces, métricas, capturas o resultados exportados. No inventar métricas si no se ejecutó la evaluación.

Aplicación en ComprasLab

Esta fase convierte ComprasLab en el sistema final: consulta datos, recupera políticas, llama APIs, mantiene la conversación, verifica sus afirmaciones y deja evidencia cuantitativa de cómo se comporta.

Fase 5 - Rúbricas que debes demostrar

R24 - ¿Construyó un mecanismo de verificación de consistencia factual y coherencia lógica de las respuestas del agente?
Evidencia esperada: Casos NLI + verificaciones exactas integradas antes de publicar.

R25 - ¿Consolidó un agente único integrando herramientas de RAG, SQL o API bajo el mismo ejecutor?
Evidencia esperada: Ejecución completa usando al menos dos categorías de herramientas.

R26 - ¿Construyó el ejecutor con create_openai_tools_agent y AgentExecutor?
Evidencia esperada: Código y ejecución del agente con esas APIs.

R27 - ¿Integró el histórico de conversación con RunnableWithMessageHistory?
Evidencia esperada: Sesión multi-turno del agente final con historial operativo.

R28 - ¿Integró el agente completo a una interfaz Streamlit?
Evidencia esperada: Interfaz ejecutable conectada al mismo agente y memoria.

R29 - ¿Evaluó calidad de respuestas y trayectoria de herramientas con LangSmith, justificando conclusiones con las métricas obtenidas?
Evidencia esperada: Experimentos LangSmith, resultados medidos y análisis comparativo.

Checklist de cierre de la fase

• El código o notebook se ejecuta desde cero con instrucciones reproducibles.

• Existe al menos una ejecución que demuestra cada rúbrica de esta fase.

• La evidencia diferencia claramente datos reales, datos dummy y resultados simulados.

• La documentación explica qué decisión técnica tomaste y por qué.

• Los pendientes o limitaciones están escritos explícitamente; no se presentan como funcionalidades terminadas.

Resumen final: relación fase → temas → rúbricas

Fase 1 - R01-R06

Temas principales: T1, T2, T3, T4, T5, T6, T17, T18, T21

Resultado que debes poder enseñar: una chain estructurada y arquitectura documentada.

Fase 2 - R07-R12

Temas principales: T6, T7, T8, T9, T10, T11, T12, T13-T16

Resultado que debes poder enseñar: un agente que elige y usa herramientas con confiabilidad observable.

Fase 3 - R13-R18

Temas principales: T14, T17, T18, T19, T20, T21

Resultado que debes poder enseñar: una conversación multi-turno con memoria y recuperación de contexto.

Fase 4 - R19-R23

Temas principales: T10, T15-T16, T22, T23, T24, T25, T26

Resultado que debes poder enseñar: consultas estructuradas + RAG sobre índice existente + corrección de una recuperación fallida.

Fase 5 - R24-R29

Temas principales: T16, T24, T26, T25, T11, T14, T20, No explícito en T1-T26

Resultado que debes poder enseñar: un agente único verificado, con historial, interfaz y evaluación en LangSmith.

Entregables finales que deben apoyarse en estas evidencias

• Código / notebooks completos: funcionales, organizados y reproducibles; con alternativas dummy cuando existan APIs o credenciales.

• Informe técnico en PDF: problema, dominio, decisiones por fase, resultados de evaluación/verificación, limitaciones, mejoras y evidencias.

• Registro de evaluación en LangSmith: calidad y trayectoria del agente integrador final.

• Video de demostración: interacción real, uso de herramienta/API externa y evidencia de memoria entre turnos.

Nota sobre discrepancias del enunciado

Hay algunos puntos donde el texto de desarrollo y la rúbrica no tienen exactamente la misma redacción (por ejemplo, Streamlit aparece como opcional en el desarrollo, pero tiene una fila propia en la rúbrica). Esta guía adopta la opción más conservadora: demostrar el requisito cuando la rúbrica lo evalúa explícitamente.

Leyenda completa de temas y rúbricas

Anexo de consulta para la guía de fases, tareas y rúbricas de ComprasLab

Cómo leer esta leyenda. T1-T26 reproducen el contenido del resumen “Temas de las clases 2 a 6”. R01-R29 reproducen, en español, lo que pregunta cada ítem de la rúbrica del proyecto. La columna de relación indica qué temas ayudan a demostrar cada rúbrica; esa relación es una guía de estudio del proyecto, no una numeración oficial del profesor.

Leyenda A - Temas T1-T26

Estos son los temas que aparecen en el resumen de las clases. Después de cada tema se indican las rúbricas de ComprasLab con las que se relaciona directamente o sirve de apoyo.

T1 · Clase 2 - Fundamentos de LangChain

Configuración de LangChain y uso de ChatOpenAI. Diferencias entre modelos de texto y de chat, roles de los mensajes y parámetros como temperature y max_tokens.

Rúbricas relacionadas: R01

T2 · Clase 2 - Fundamentos de LangChain

Plantillas reutilizables con PromptTemplate y ChatPromptTemplate para insertar variables como el contexto y la pregunta.

Rúbricas relacionadas: R03

T3 · Clase 2 - Fundamentos de LangChain

Salidas estructuradas con Pydantic y with_structured_output(). Conversión de texto separado por comas en listas de Python.

Rúbricas relacionadas: R04

T4 · Clase 2 - Fundamentos de LangChain

Métodos de Runnable: invoke() para una entrada, batch() para un lote y stream() para recibir fragmentos a medida que se generan.

Rúbricas relacionadas: R05

T5 · Clase 2 - Fundamentos de LangChain

Composición de cadenas con funciones decoradas con @chain y con el operador | de LCEL.

Rúbricas relacionadas: R05, R06

T6 · Clase 3 - Construcción de agentes avanzados

Orquestación de modelos, herramientas y memoria. Diferencia entre un flujo predefinido y un agente que decide qué herramientas usar.

Rúbricas relacionadas: R01, R02, R08

T7 · Clase 3 - Construcción de agentes avanzados

Comparación de LangChain y LangGraph, LlamaIndex, AutoGen, Semantic Kernel y Langflow.

Rúbricas relacionadas: R07

T8 · Clase 3 - Construcción de agentes avanzados

Cadenas secuenciales y procesamiento en paralelo. El ejercicio encadenó el resumen de una reseña, la identificación de problemas y la propuesta de mejoras.

Rúbricas relacionadas: R09

T9 · Clase 3 - Construcción de agentes avanzados

Enrutamiento dinámico para elegir entre una respuesta para principiantes y una explicación avanzada.

Rúbricas relacionadas: R09

T10 · Clase 3 - Construcción de agentes avanzados

Integración de APIs mediante @tool y function calling, con Open-Meteo como ejemplo.

Rúbricas relacionadas: R10, R20, R25

T11 · Clase 3 - Construcción de agentes avanzados

Construcción de un agente ReAct con create_agent y memoria de sesión con MemorySaver y thread_id.

Rúbricas relacionadas: R08, R16, R26, R27

T12 · Clase 4 - Prompts, memoria y búsqueda semántica

Diseño de prompts few-shot para clasificar reseñas de películas usando ejemplos etiquetados.

Rúbricas relacionadas: R11, R17

T13 · Clase 4 - Prompts, memoria y búsqueda semántica

Validación de respuestas con PydanticOutputParser y reparación de salidas malformadas mediante otra cadena.

Rúbricas relacionadas: R04, R12

T14 · Clase 4 - Prompts, memoria y búsqueda semántica

Gestión del historial con un buffer completo, un límite de tokens o un resumen de los mensajes antiguos.

Rúbricas relacionadas: R08, R12, R14, R16, R27

T15 · Clase 4 - Prompts, memoria y búsqueda semántica

Creación de embeddings, indexación con FAISS y búsqueda por similitud con filtros de metadatos.

Rúbricas relacionadas: R12, R22

T16 · Clase 4 - Prompts, memoria y búsqueda semántica

Uso de callbacks para registrar llamadas al modelo y a sus herramientas. Integración de FAISS como herramienta de un agente ReAct.

Rúbricas relacionadas: R12, R22, R29

T17 · Clase 5 - Memoria y flujo conversacional

Memoria de corto plazo, caché semántica y memoria de largo plazo, dividida en semántica, episódica y procedimental. El ejercicio guardó una solución de Spark y la recuperó en otra sesión.

Rúbricas relacionadas: R01, R02, R13

T18 · Clase 5 - Memoria y flujo conversacional

Gestión de la ventana de contexto con filtros de relevancia y recorte de tokens. Recuperación de recuerdos por similitud, antigüedad y frecuencia de acceso, y actualización de recuerdos obsoletos.

Rúbricas relacionadas: R02, R14

T19 · Clase 5 - Memoria y flujo conversacional

Comparación de una implementación propia con Mem0 y Letta/MemGPT. Ejercicio inspirado en LangMem para guardar preferencias durante la conversación.

Rúbricas relacionadas: R15

T20 · Clase 5 - Memoria y flujo conversacional

Reconocimiento de intenciones y extracción de datos como destino, fecha y número de pasajeros. Solicitud de datos faltantes y reutilización de lo dicho en turnos anteriores.

Rúbricas relacionadas: R16, R17, R18, R27

T21 · Clase 5 - Memoria y flujo conversacional

División de solicitudes complejas en varias acciones. El ejercicio simuló búsquedas de vuelos y hoteles, consultas de saldo, pagos y una transferencia.

Rúbricas relacionadas: R02, R11, R18

T22 · Clase 6 - Integración y diagnóstico de agentes

Herramientas deterministas para cálculos, semánticas para analizar informes y asíncronas para consultar varios clústeres en paralelo.

Rúbricas relacionadas: R09, R19, R25

T23 · Clase 6 - Integración y diagnóstico de agentes

Conversión de preguntas en lenguaje natural a consultas SQL de lectura sobre una base SQLite con empleados y proyectos.

Rúbricas relacionadas: R21, R25

T24 · Clase 6 - Integración y diagnóstico de agentes

RAG con FAISS. División de documentos en fragmentos solapados, reformulación de consultas, filtros por dominio y diagnóstico mediante el documento de origen, el identificador del fragmento y la distancia L2.

Rúbricas relacionadas: R22, R23, R24, R25

T25 · Clase 6 - Integración y diagnóstico de agentes

Integración de SQL, FAISS, conversión de moneda, telemetría y síntesis de informes en un agente ReAct con LangGraph. El ejercicio cruzó datos de un empleado con la política de viajes y calculó sus viáticos.

Rúbricas relacionadas: R20, R25, R26, R29

T26 · Clase 6 - Integración y diagnóstico de agentes

Diagnóstico de búsquedas fallidas y respuesta del agente cuando los documentos recuperados no respaldan una respuesta.

Rúbricas relacionadas: R23, R24

Leyenda B - Rúbricas R01-R29

La rúbrica oficial está organizada en cinco fases. A continuación se presenta el sentido de cada ítem en español y los temas de clase que debes estudiar para demostrarlo. Cuando un requisito exige una API o clase que no aparece literalmente en el resumen de temas, se señala con una nota.

Fase 1

R01

¿El alumno estructuró la anatomía y la arquitectura cognitiva del agente, clasificando su tipo y nivel de madurez agéntica?

Temas relacionados: T1, T6, T17

R02

¿El alumno proyectó una arquitectura de planificación y memoria aumentada para el agente?

Temas relacionados: T6, T17, T18, T21

R03

¿El alumno construyó plantillas de prompt reutilizables con PromptTemplate o ChatPromptTemplate?

Temas relacionados: T2

R04

¿El alumno implementó un output parser para generar salida estructurada en JSON a partir de la chain?

Temas relacionados: T3, T13

R05

¿El alumno aplicó la interfaz Runnable (invoke, batch, stream) e implementó composición imperativa con el decorador @chain?

Temas relacionados: T4, T5

R06

¿El alumno implementó la composición declarativa de la chain con LCEL y justificó la elección entre composición imperativa y declarativa?

Temas relacionados: T5

Fase 2

R07

¿El alumno comparó orquestadores de agentes autónomos y justificó la elección de LangChain para el problema?

Temas relacionados: T7

R08

¿El alumno construyó un agente con herramientas y memoria e implementó el patrón ReAct con initialize_agent?

Temas relacionados: T6, T11, T14

Nota: el resumen de clase menciona create_agent; la rúbrica pide demostrar initialize_agent.

R09

¿El alumno construyó chains secuenciales o paralelas e implementó enrutamiento dinámico con RouterChain?

Temas relacionados: T8, T9, T22

Nota: el tema T9 explica enrutamiento dinámico; la rúbrica nombra específicamente RouterChain.

R10

¿El alumno integró por lo menos una API externa como herramienta del agente vía function calling?

Temas relacionados: T10

R11

¿El alumno estructuró el prompt del agente con el framework PTCF y aplicó few-shot learning y chain-of-thought?

Temas relacionados: T12, T21

R12

¿El alumno implementó los componentes de confiabilidad del agente (output parsers con tratamiento de errores, memoria configurada en la chain, herramienta de búsqueda semántica con FAISS y callbacks de logging), justificando las decisiones técnicas?

Temas relacionados: T13, T14, T15, T16

Fase 3

R13

¿El alumno diferenció memoria de corto y largo plazo para el agente con base en la taxonomía CoALA?

Temas relacionados: T17

R14

¿El alumno gestionó ventanas de contexto combinando sliding window o summarization con búsqueda vectorial?

Temas relacionados: T14, T18

R15

¿El alumno comparó herramientas especializadas de memoria por arquitectura y caso de uso?

Temas relacionados: T19

R16

¿El alumno construyó un chatbot conversacional con ConversationBufferMemory?

Temas relacionados: T11, T14, T20

Nota: los temas explican memoria conversacional; la rúbrica pide específicamente ConversationBufferMemory.

R17

¿El alumno estructuró flujos de conversación con reconocimiento de intención y extracción de entidades?

Temas relacionados: T20, T12

R18

¿El alumno validó, con evidencia de una conversación real, que el agente mantiene contexto y descompone consultas complejas en diálogos multi-turno?

Temas relacionados: T20, T21

Fase 4

R19

¿El alumno estructuró herramientas del agente diferenciando funciones hardcoded de funciones semánticas?

Temas relacionados: T22

R20

¿El alumno integró el agente a APIs internas o externas de acuerdo con el tipo de uso de cada una?

Temas relacionados: T10, T25

R21

¿El alumno implementó el patrón text-to-query para consultar datos estructurados?

Temas relacionados: T23

R22

¿El alumno construyó un agente de recuperación usando un índice FAISS existente?

Temas relacionados: T15, T16, T24

R23

¿El alumno documentó una falla real de recuperación a partir de los documentos fuente retornados y justificó la técnica de corrección aplicada?

Temas relacionados: T24, T26

Fase 5

R24

¿El alumno construyó un mecanismo de verificación de consistencia factual y coherencia lógica de las respuestas del agente?

Temas relacionados: T26, T24

Nota: la fase final del enunciado añade verificación factual/coherencia; no aparece como tema independiente entre T1-T26.

R25

¿El alumno consolidó un agente único integrando herramientas de RAG, SQL o API bajo un mismo executor?

Temas relacionados: T25, T10, T22, T23, T24

R26

¿El alumno construyó el executor del agente con create_openai_tools_agent y AgentExecutor?

Temas relacionados: T11, T25

Nota: la rúbrica exige específicamente create_openai_tools_agent y AgentExecutor.

R27

¿El alumno integró el histórico de conversación al agente con RunnableWithMessageHistory?

Temas relacionados: T11, T14, T20

Nota: la rúbrica exige específicamente RunnableWithMessageHistory.

R28

¿El alumno integró el agente completo a una interfaz Streamlit?

Temas relacionados: No tiene un tema T1-T26 equivalente directo.

Nota: Streamlit no aparece en el resumen T1-T26; es un requisito adicional de la rúbrica.

R29

¿El alumno evaluó la calidad de las respuestas y la trayectoria de llamadas de herramientas del agente con LangSmith, justificando las conclusiones a partir de las métricas obtenidas?

Temas relacionados: T16, T25

Nota: callbacks/telemetría apoyan la observabilidad, pero la rúbrica exige evaluación en LangSmith.

Mapa inverso - de la rúbrica al tema que debes estudiar

Rúbrica	Fase	Temas
R01	1	T1, T6, T17
R02	1	T6, T17, T18, T21
R03	1	T2
R04	1	T3, T13
R05	1	T4, T5
R06	1	T5
R07	2	T7
R08	2	T6, T11, T14
R09	2	T8, T9, T22
R10	2	T10
R11	2	T12, T21
R12	2	T13, T14, T15, T16
R13	3	T17
R14	3	T14, T18
R15	3	T19
R16	3	T11, T14, T20
R17	3	T20, T12
R18	3	T20, T21
R19	4	T22
R20	4	T10, T25
R21	4	T23
R22	4	T15, T16, T24
R23	4	T24, T26
R24	5	T26, T24
R25	5	T25, T10, T22, T23, T24
R26	5	T11, T25
R27	5	T11, T14, T20
R28	5	Requisito adicional
R29	5	T16, T25

Fuentes. Temas: documento “Temas de las clases 2 a 6”, 26 puntos sobre el desarrollo de agentes cognitivos. Rúbricas: páginas 6 a 8 del enunciado “Agentes Cognitivos com LangChain [26E4_2]”. La traducción al español mantiene el sentido de las preguntas de la rúbrica; los códigos T y R son etiquetas de esta guía.